



Université de Tunis El Manar
Faculté des Sciences de Tunis
Département Informatique — 4ème Année Cycle Ingénieur

RAPPORT DE STAGE

SecureZone

Plateforme unifiée SIEM / SOAR / Vulnerability Management

Étudiante
Entreprise d'accueil
Filière
Année universitaire

Ben Ghazi Hajer
MAE
Computer Engineering (ICE4)
2025–2026

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce stage.

Je remercie particulièrement mon encadrant au sein de MAE pour son accompagnement, sa disponibilité et ses précieux conseils tout au long de ce stage. Ses orientations et son soutien m'ont été d'une grande aide dans la réalisation de ce travail.

J'adresse également mes sincères remerciements aux enseignants de la Faculté des Sciences de Tunis pour la qualité de la formation qu'ils m'ont assurée tout au long de mon parcours académique.

Résumé

Ce rapport présente SecureZone, une plateforme de cybersécurité unifiée développée dans le cadre d'un stage au sein de MAE, une compagnie d'assurance soumise à la réglementation européenne DORA. La plateforme intègre quatre modules complémentaires : un SIEM pour la collecte et l'analyse des logs avec détection d'anomalies par Machine Learning (Isolation Forest) et corrélation par règles à fenêtre glissante, un module de détection de phishing basé sur un système de score heuristique, un module de gestion des vulnérabilités (VM) orchestrant des scans réseau et de vulnérabilités réels via Nmap et OpenVAS, un module de conformité évaluant automatiquement les exigences DORA/ISO 27001/CIS Controls, et un module de réponse aux incidents (IR) avec orchestration SOAR. Le backend repose sur FastAPI, SQLAlchemy 2.0 et PostgreSQL, le frontend sur React. Un assistant IA (SecureBot, basé sur Groq/Llama 3.1) fournit des réponses contextuelles aux analystes. Le module SIEM/SOAR a été validé avec quatre playbooks (Brute Force, Phishing, Port Scan, Generic High Severity), et le module VM a été testé en conditions réelles sur un environnement Metasploitable2, révélant 363 vulnérabilités.

Mots-clés : SIEM, SOAR, Vulnerability Management, DORA, FastAPI, Machine Learning

Table des matières

Remerciements	1
Résumé	2
Liste des Abréviations	6
Introduction Générale	7
1 Introduction et Contexte	8
1.1 Contexte du Stage	9
1.2 Problématique	9
1.3 Étude et Critique de l’Existant	9
1.4 Solution Proposée	9
1.5 Objectifs du Stage	9
2 Préparation du Projet et Analyse des Besoins	11
2.1 Identification des Acteurs	12
2.2 Spécification des Besoins	12
2.2.1 Besoins Fonctionnels	12
2.2.2 Besoins Non Fonctionnels	12
2.3 Technologies Utilisées	12
3 Conception du Système	14
3.1 Architecture Globale	15
3.2 Architecture du Module SIEM	15
3.3 Architecture du Module Phishing	15
3.4 Architecture du Module VM	16
3.5 Architecture du Module Compliance	17
3.6 Architecture du Module Incident Response et SOAR	17
4 Réalisation, Déploiement et Tests	19
4.1 Interfaces Réalisées	20
4.2 Assistant IA SecureBot	21
4.3 Intégration OpenVAS	21
4.4 Tests et Validation	22
5 Limites et Perspectives	24
5.1 Limites Actuelles	25
5.2 Perspectives	25
Conclusion Générale	26

Table des figures

4.1	Dashboard principal de SecureZone	20
4.2	Page Alertes SIEM avec alerte corrélée Brute Force	20
4.3	Analyse d'URL par le module Phishing	21
4.4	Page Vulnérabilités – résultats d'un scan OpenVAS	21
4.5	Tableau de conformité ISO 27001 / DORA / CIS Controls	22
4.6	Détail d'un incident avec playbook SOAR attaché	22
4.7	Interaction avec l'assistant SecureBot	23

Liste des tableaux

2.1	Besoins fonctionnels du système SecureZone	12
2.2	Besoins non fonctionnels du système SecureZone	13
2.3	Stack technique complète de SecureZone	13
3.1	Modules de SecureZone	15
3.2	Règles de corrélation du moteur SIEM	16
3.3	Heuristiques de détection de phishing	16
3.4	Playbooks SOAR de SecureZone	17
4.1	Endpoints du module Vulnerability Management	23
4.2	Résultats des tests de validation	23
5.1	Limites actuelles de SecureZone	25

Liste des Abréviations

SIEM Security Information and Event Management

SOAR Security Orchestration, Automation and Response

VM Vulnerability Management

IR Incident Response

DORA Digital Operational Resilience Act

CVE Common Vulnerabilities and Exposures

CVSS Common Vulnerability Scoring System

JWT JSON Web Token

API Application Programming Interface

MITRE ATT&CK Référentiel des tactiques et techniques d'attaque

RAG Retrieval-Augmented Generation

ISO 27001 Norme internationale de gestion de la sécurité de l'information

CIS Controls Center for Internet Security Controls

Introduction Générale

La cybersécurité est devenue un enjeu stratégique majeur pour les institutions financières, en particulier depuis l'entrée en vigueur du règlement européen DORA (Digital Operational Resilience Act), qui impose aux compagnies d'assurance et autres acteurs financiers de démontrer une résilience opérationnelle numérique continue.

Face à ce constat, les approches traditionnelles restent fragmentées : les outils de surveillance (SIEM), de gestion des vulnérabilités et de suivi de conformité sont généralement cloisonnés, ce qui complique la vision globale de la posture de sécurité d'une organisation.

C'est dans ce contexte que s'inscrit le stage à l'origine de ce rapport, réalisé au sein de MAE. Ce stage a donné lieu au développement de SecureZone, une plateforme de cybersécurité unifiée intégrant SIEM, gestion des vulnérabilités, conformité réglementaire et réponse aux incidents au sein d'une interface unique.

Ce rapport est organisé comme suit :

- Le **Chapitre 1** présente le contexte du stage, la problématique et les objectifs.
- Le **Chapitre 2** détaille l'analyse des besoins et les technologies retenues.
- Le **Chapitre 3** expose la conception globale du système.
- Le **Chapitre 4** regroupe la réalisation, le déploiement et les tests de validation.
- Le **Chapitre 5** présente les limites actuelles et les perspectives d'évolution.
- Enfin, une **conclusion générale** synthétise les apports du stage.

Chapitre 1

Introduction et Contexte

Plan

Introduction

1.1 Contexte du Stage

1.2 Problématique

1.3 Étude et Critique de l'Existant

1.4 Solution Proposée

1.5 Objectifs du Stage

Conclusion

Introduction

Ce chapitre présente le cadre général du stage, la problématique de sécurité à laquelle répond le projet SecureZone, les limites des solutions existantes, ainsi que les objectifs fixés pour ce travail.

1.1 Contexte du Stage

Ce stage a été réalisé au sein de MAE, une compagnie d'assurance soumise à la réglementation européenne DORA, qui impose aux institutions financières de démontrer leur résilience opérationnelle numérique face aux cybermenaces. Dans ce cadre, l'entreprise a besoin d'outils capables de centraliser la surveillance de sécurité, la gestion des vulnérabilités et le suivi de conformité réglementaire.

1.2 Problématique

La gestion de la cybersécurité au sein d'une organisation repose généralement sur plusieurs outils indépendants : un SIEM pour la collecte et l'analyse des logs, un scanner de vulnérabilités pour identifier les failles, et des tableaux de bord de conformité gérés manuellement. Cette fragmentation pose plusieurs limites :

- Absence de vue unifiée entre les alertes de sécurité, les vulnérabilités et l'état de conformité réglementaire.
- Traitement manuel et chronophage des incidents de sécurité.
- Difficulté à démontrer la conformité DORA de manière continue et automatisée.

Il manque donc une plateforme intégrée capable de croiser ces informations pour offrir une vision cohérente de la posture de sécurité de l'entreprise.

1.3 Étude et Critique de l'Existant

Plusieurs solutions du marché répondent partiellement à ce besoin :

- **Splunk / IBM QRadar** : SIEM commerciaux puissants, mais coûteux et fermés, avec une intégration limitée de la gestion des vulnérabilités et de la conformité réglementaire spécifique.
- **Wazuh (seul)** : solution SIEM open-source performante pour la détection, mais ne couvre ni la gestion des vulnérabilités (VM) ni le suivi de conformité.
- **OpenVAS / Nessus (seuls)** : efficaces pour le scan de vulnérabilités, mais sans corrélation avec les événements de sécurité ni tableau de conformité.

Aucune de ces solutions, prise isolément, ne permet de croiser les trois dimensions (SIEM, VM, Compliance) au sein d'une interface unique.

1.4 Solution Proposée

SecureZone est une plateforme de cybersécurité unifiée et open-source, organisée en quatre modules :

- **SIEM** : collecte, normalisation, corrélation des logs et détection d'anomalies par Machine Learning (Isolation Forest).
- **VM (Vulnerability Management)** : orchestration de scans réseau (Nmap) et de vulnérabilités (OpenVAS/GVM), calcul d'un score de risque par actif.
- **Compliance** : évaluation automatique des contrôles DORA en croisant les données des autres modules.
- **IR (Incident Response)** : gestion des tickets d'incidents et orchestration SOAR.

1.5 Objectifs du Stage

Les objectifs fixés pour ce stage sont :

- Concevoir et développer une architecture backend (FastAPI) et frontend (React) modulaire et sécurisée.
- Intégrer un moteur de détection d'anomalies par apprentissage non supervisé (Isolation Forest) sur les logs collectés via Wazuh.

- Développer un moteur de gestion des vulnérabilités intégrant un scanner réel (OpenVAS) sur un environnement de test (Metasploitable2).
- Automatiser l'évaluation de la conformité DORA en croisant les données SIEM et VM.

Conclusion

Ce chapitre a présenté le contexte du stage et la problématique de fragmentation des outils de cybersécurité au sein d'une compagnie d'assurance soumise à DORA. Le chapitre suivant détaille l'analyse des besoins et les choix technologiques retenus pour SecureZone.

Chapitre 2

Préparation du Projet et Analyse des Besoins

Plan

Introduction

2.1 Identification des Acteurs

2.2 Spécification des Besoins

2.3 Technologies Utilisées

Conclusion

Introduction

Ce chapitre présente les acteurs du système, les besoins fonctionnels et non fonctionnels identifiés, ainsi que les technologies retenues pour la réalisation de SecureZone.

2.1 Identification des Acteurs

Le système interagit avec les acteurs suivants :

- **L’analyste SOC** : consulte les alertes SIEM, déclenche les playbooks de réponse et suit les tickets d’incidents.
- **Le RSSI (Responsable de la Sécurité des Systèmes d’Information)** : supervise le tableau de bord de conformité DORA/ISO 27001/CIS Controls.
- **L’administrateur** : gère les scans de vulnérabilités et les actifs surveillés.

2.2 Spécification des Besoins

2.2.1 Besoins Fonctionnels

Réf.	Besoin fonctionnel	Priorité
BF-01	Collecte et normalisation des logs via Wazuh	Haute
BF-02	Détection d’anomalies par Machine Learning (Isolation Forest)	Haute
BF-03	Corrélation des événements de sécurité (6 règles)	Haute
BF-04	Exécution automatique de playbooks SOAR (Brute Force, Phishing, Port Scan, Generic High Severity)	Haute
BF-05	Découverte réseau et scan de vulnérabilités (Nmap + OpenVAS)	Haute
BF-06	Calcul d’un score de risque par actif	Haute
BF-07	Import des vulnérabilités détectées par un scanner externe	Haute
BF-08	Évaluation automatique des contrôles de conformité DORA	Haute
BF-09	Tableau de bord de conformité ISO 27001 / DORA / CIS Controls	Haute
BF-10	Gestion des tickets d’incidents (IR)	Moyenne
BF-11	Authentification sécurisée (JWT)	Haute
BF-12	Analyse d’URLs et détection de phishing	Haute
BF-13	Assistant IA contextuel (SecureBot)	Moyenne
BF-14	Génération de rapports PDF (exécutif, technique, conformité)	Moyenne

TABLE 2.1 – Besoins fonctionnels du système SecureZone

2.2.2 Besoins Non Fonctionnels

2.3 Technologies Utilisées

Conclusion

Ce chapitre a permis d’identifier les acteurs, de formaliser les besoins fonctionnels et non fonctionnels, et de justifier les choix technologiques. Le chapitre suivant présente la conception globale du système.

Exigence	Description	Cible
Sécurité	Authentification JWT signée (HS256)	Tokens 60 min
Fiabilité	Absence de doublons lors de la persistance des vulnérabilités	–
Scalabilité	Base de données relationnelle robuste	PostgreSQL 16
Automatisation	Scans et collecte planifiés sans intervention manuelle	APScheduler
Conformité	Traçabilité des contrôles réglementaires	DORA / ISO 27001 / CIS
Portabilité	Architecture prête à être conteneurisée	Docker Compose (cible)

TABLE 2.2 – Besoins non fonctionnels du système SecureZone

Domaine	Technologie	Détail
Backend	FastAPI 0.111	Routes asynchrones
ORM	SQLAlchemy 2.0	Accès BD asynchrone
Base de données	PostgreSQL 16	Via asyncpg 0.29
Authentification	python-jose (JWT)	HS256, tokens 60 min
Tâches planifiées	APScheduler	Cron des scans, collecte toutes les 60s
Frontend	React + Vite 5.x	SPA
Requêtes/état	TanStack Query v5	Polling serveur
Style	Tailwind CSS 3.x	Design system
Rapports PDF	ReportLab 4.2	Génération de rapports
Détection d'anomalies	scikit-learn 1.4 (Isolation Forest)	Apprentissage non supervisé
Assistant IA	Groq API (llama-3.1-8b-instant)	Chatbot SecureBot
Collecteur de logs	API REST Wazuh 4.x	Agents SIEM
Scanner de vulnérabilités	Nmap + OpenVAS/GVM	Détection de CVE

TABLE 2.3 – Stack technique complète de SecureZone

Chapitre 3

Conception du Système

Plan

Introduction

3.1 Architecture Globale

3.2 Architecture du Module SIEM

3.3 Architecture du Module Phishing

3.4 Architecture du Module VM

3.5 Architecture du Module Compliance

3.6 Architecture du Module Incident Response et SOAR

Conclusion

Introduction

Ce chapitre présente l’architecture globale de SecureZone ainsi que le fonctionnement détaillé de chacun de ses modules : SIEM, Phishing, VM, Compliance et Incident Response/SOAR.

3.1 Architecture Globale

SecureZone repose sur une architecture modulaire à composants interconnectés :

Module	Rôle	Statut
SIEM	Collecte, corrélation et détection d’anomalies	Complet
Phishing	Analyse d’URLs et d’e-mails suspects	Complet
VM	Gestion des vulnérabilités (scan, import, scoring)	Complet
Compliance	Vérification automatique de la conformité DORA	Complet
IR / SOAR	Gestion des incidents et orchestration de la réponse	Complet

TABLE 3.1 – Modules de SecureZone

Le backend expose une API REST via FastAPI, consommée par le frontend React. Les modules SIEM et VM communiquent avec des outils externes (Wazuh, Nmap, OpenVAS) tandis que le module Compliance interroge les autres modules pour établir un score de conformité global, et le module IR/SOAR réagit automatiquement aux alertes critiques générées par le SIEM et le module Phishing.

3.2 Architecture du Module SIEM

Le pipeline SIEM se déroule en quatre étapes séquentielles :

collecte(Wazuh) → normalisation → détection d’anomalies(ML) → corrélation → persistance

La détection d’anomalies repose sur un modèle Isolation Forest, un algorithme de Machine Learning non supervisé, appliqué à un vecteur de 11 caractéristiques par événement (heure, jour de la semaine, sévérité, port, catégorie encodée, octets de l’IP source, indicateurs hors-heures/week-end). Le modèle est ré-entraîné toutes les 6 heures sur les 2000 derniers événements et nécessite au minimum 100 échantillons pour s’activer ; en deçà, les événements passent sans score. Cette approche non supervisée est particulièrement adaptée au contexte de la cybersécurité, où l’on ne dispose pas toujours d’exemples labellisés d’attaques.

Chaque événement normalisé est également étiqueté avec sa technique **MITRE ATT&CK** correspondante (par exemple T1110 pour le Brute Force, T1595 pour le Scan Actif), une pratique standard des SIEM d’entreprise qui facilite l’analyse et le reporting.

Le moteur de corrélation applique six règles à fenêtre glissante en mémoire, indexées par adresse IP source :

Après déclenchement d’une règle, la fenêtre de l’IP concernée est réinitialisée afin d’éviter une cascade d’alertes issues du même pic d’événements. Une fois une alerte critique ou haute générée, le module Incident Response peut déclencher automatiquement un playbook SOAR.

3.3 Architecture du Module Phishing

Le module de détection de phishing analyse les URLs à l’aide d’un système de score cumulatif (0 à 100), combinant six heuristiques indépendantes :

Règle	Nom	Seuil	Fenêtre	Condition
CR-001	Brute Force	5 événements	60s	Échecs d'authentification répétés
CR-002	Port Scan	10 événements	120s	Événements de reconnaissance
CR-003	Multi-Target	5 événements	300s	5+ destinations différentes
CR-004	Exploit	3 événements	60s	Attaques web/exploit
CR-005	Credentials	3 événements	300s	Événements liés aux identifiants
CR-006	Lateral Movement	3 événements	600s	Mouvement latéral interne

TABLE 3.2 – Règles de corrélation du moteur SIEM

Vérification	Points	Condition
Typosquatting	+40	Distance d'édition ≤ 2 avec une marque connue
Homoglyphe	+40	Caractères Unicode similaires (ex. cyrillique)
TLD suspect	+25	Extensions .tk, .ml, .ga, .cf, .gq, .xyz...
Entropie élevée	+20	Entropie de Shannon ≥ 3.5
Sous-domaines excessifs	+20	Plus de 3 niveaux de sous-domaines
Hôte en IP brute	+30	L'URL utilise une adresse IP au lieu d'un nom

TABLE 3.3 – Heuristiques de détection de phishing

Le verdict final est déterminé par seuil : 0–30 = SAFE, 31–60 = SUSPICIOUS, 61+ = PHISHING. Une URL classée PHISHING déclenche automatiquement une alerte SIEM de catégorie *phishing*, pouvant elle-même activer le playbook SOAR correspondant.

3.4 Architecture du Module VM

Le module de gestion des vulnérabilités (VM Engine) orchestre un pipeline de scan en cinq phases :

1. Création d'un `ScanJob` (statut *pending*).
2. Découverte des hôtes actifs et des ports ouverts via Nmap.
3. Mise à jour des actifs (*Assets*) en base de données.
4. Détection des CVE sur chaque hôte via OpenVAS.
5. Persistance des vulnérabilités sans doublons et recalcul du score de risque de chaque actif.

Le score de risque de chaque actif est calculé selon la formule suivante, pondérant chaque vulnérabilité par sa sévérité et son score CVSS, puis ajustée par un multiplicateur de criticité de l'actif :

Listing 3.1 – Formule de calcul du `risk_score`

```

1 SEVERITY_WEIGHTS = {
2     "critical": 4.0,
3     "high": 2.5,
4     "medium": 1.0,
5     "low": 0.3
6 }
7 CRITICALITY_MULTIPLIER = {
8     "critical": 2.0,
9     "high": 1.5,
```

```

10     "medium": 1.0,
11     "low": 0.5
12 }
13
14 base = sum(
15     SEVERITY_WEIGHTS[sev] * cvss_score / 10
16     for sev, cvss_score in open_vulnerabilities
17 )
18 risk_score = min(10.0, base) * CRITICALITY_MULTIPLIER[asset_criticality]

```

Ce module a été validé en conditions réelles : des scans OpenVAS/Nessus menés contre une machine Metasploitable2 ont permis d’identifier 363 vulnérabilités, confirmant la capacité du système à traiter des données réelles à grande échelle.

3.5 Architecture du Module Compliance

Le moteur de conformité s’appuie sur 11 types de règles de politique, couvrant des vérifications statiques (ports ouverts, version logicielle, chiffrement...) et trois règles connectées en temps réel aux alertes SIEM : absence de brute force actif, absence de détection de phishing, et absence d’alerte critique ouverte sur l’actif. Par exemple :

- Le contrôle « *Aucun brute force actif* » interroge le module SIEM et échoue si une alerte de type brute force est ouverte sur l’actif.
- Le contrôle « *Patch appliqué* » interroge le module VM et échoue si des CVE critiques/hautes restent ouvertes.

Le score de conformité d’un actif est calculé ainsi :

$$score_{actif} = \frac{nb_{conforme} + 0.5 \times nb_{partiel}}{nb_{applicable}} \times 100$$

Le score global est la moyenne des scores de tous les actifs, présenté sur un tableau de bord couvrant les référentiels DORA, ISO 27001 et CIS Controls.

3.6 Architecture du Module Incident Response et SOAR

Chaque incident suit un cycle de vie structuré :

NEW → *ASSIGNED* → *INVESTIGATING* → *CONTAINMENT* → *ERADICATION* → *RECOVERY* → *CLOSE*

Quatre playbooks sont configurés, combinant des étapes automatiques (AUTO) et des étapes nécessitant l’approbation d’un analyste (MANUAL) :

Playbook	Étapes principales
Réponse Brute Force	log_ioc (AUTO), trigger_scan (AUTO), block_ip (MANUAL), reset identifiants (MANUAL)
Réponse Phishing	log_ioc (AUTO), trigger_scan (AUTO), squid_blacklist (MANUAL), notification (AUTO), sensibilisation (MANUAL)
Réponse Port Scan	log_ioc (AUTO), run_compliance (AUTO), identification (MANUAL), block_ip (MANUAL)
Générique Haute Sévérité	log_ioc (AUTO), trigger_scan (AUTO), investigation (MANUAL), confinement (MANUAL)

TABLE 3.4 – Playbooks SOAR de SecureZone

Les actions MANUAL (`block_ip`, `squid_blacklist`) ne sont pas exécutées directement : elles affichent les commandes exactes (iptables, Squid) que l’analyste doit valider et exécuter lui-même, un choix de conception

délibéré préservant le contrôle humain sur les changements réseau. Le *Mean Time To Respond* (MTTR) est calculé automatiquement à la clôture de chaque incident, à partir des horodatages de détection et de fermeture.

Conclusion

Ce chapitre a détaillé l'architecture globale de SecureZone ainsi que le fonctionnement interne de ses cinq modules : SIEM, Phishing, VM, Compliance et IR/SOAR. Le chapitre suivant présente la réalisation technique, le déploiement et les tests de validation.

Chapitre 4

Réalisation, Déploiement et Tests

Plan

Introduction

4.1 Interfaces Réalisées

4.2 Assistant IA SecureBot

4.3 Intégration OpenVAS

4.4 Tests et Validation

Conclusion

Introduction

Ce chapitre présente les interfaces réalisées, l'assistant IA intégré, l'intégration du scanner OpenVAS au sein de SecureZone, ainsi que les résultats des tests de validation menés sur la plateforme.

4.1 Interfaces Réalisées

Le frontend React de SecureZone propose un ensemble de pages permettant de piloter l'ensemble des modules : authentification, tableau de bord principal, gestion des alertes SIEM, analyse de phishing, gestion des vulnérabilités, tableau de conformité et gestion des incidents.

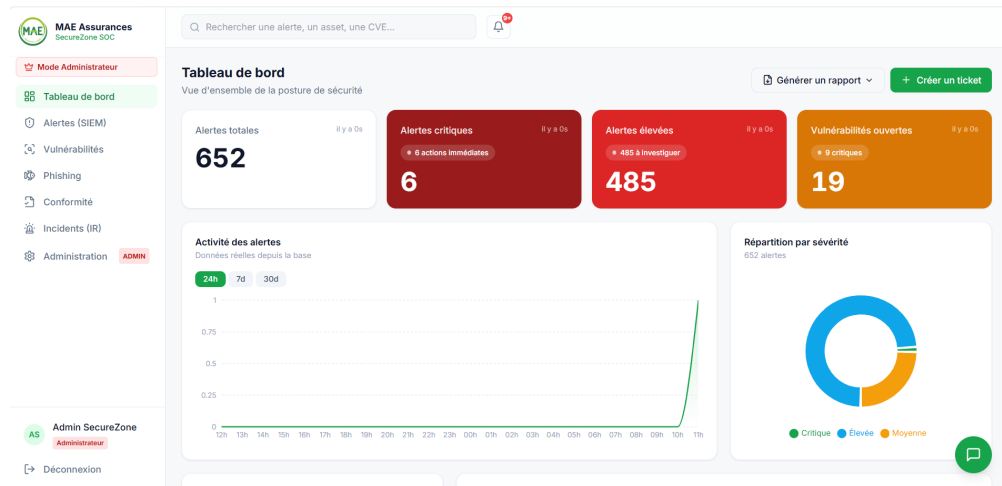


FIGURE 4.1 – Dashboard principal de SecureZone

Le tableau de bord principal centralise les indicateurs clés : nombre d'alertes actives, score de risque global des actifs, et score de conformité DORA/ISO 27001/CIS Controls (voir Figure 4.1).

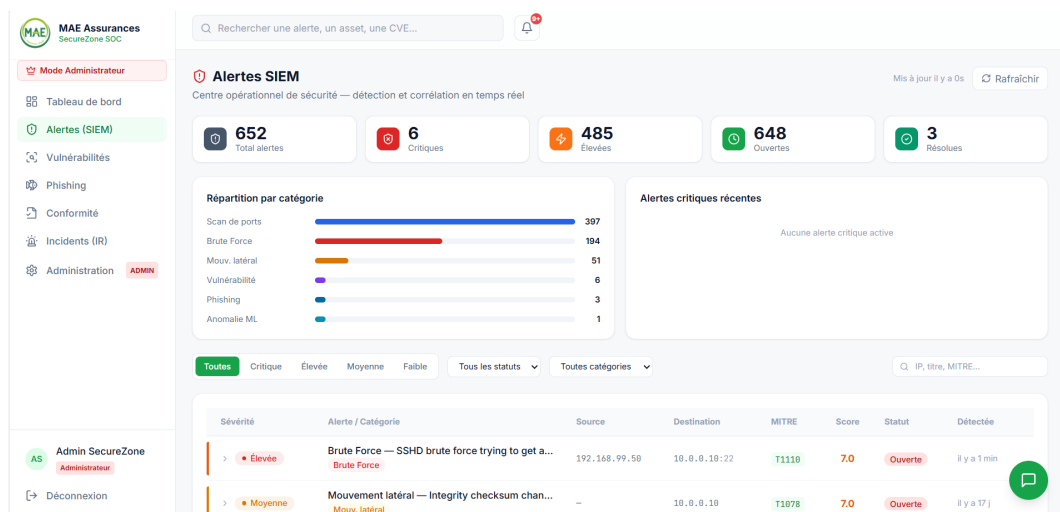


FIGURE 4.2 – Page Alertes SIEM avec alerte corrélée Brute Force

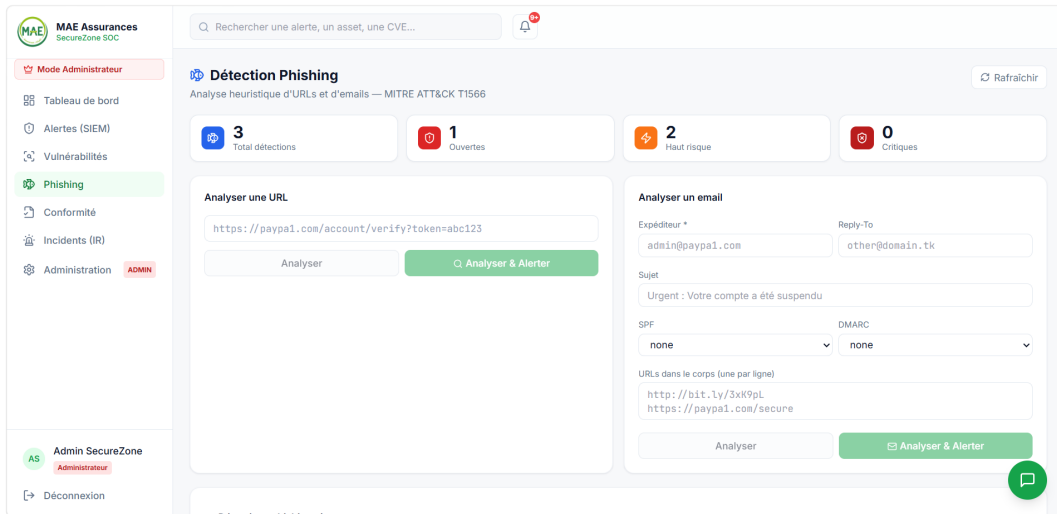


FIGURE 4.3 – Analyse d’URL par le module Phishing

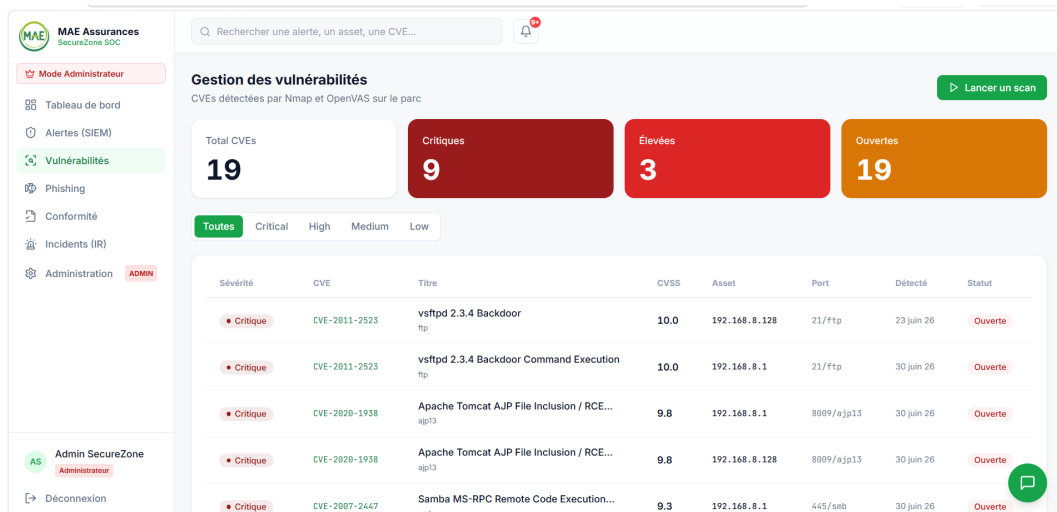


FIGURE 4.4 – Page Vulnérabilités – résultats d’un scan OpenVAS

4.2 Assistant IA SecureBot

SecureBot est un assistant conversationnel intégré au dashboard, basé sur l’API Groq (modèle `llama-3.1-8b-instant`). Avant chaque réponse, le backend interroge la base de données pour injecter le contexte de sécurité courant dans le prompt système : nombre d’alertes critiques, dernières alertes, score de conformité global, nombre de vulnérabilités ouvertes. Cette approche, proche du *Retrieval-Augmented Generation* (RAG), permet à l’analyste de poser des questions en langage naturel sans écrire de requêtes SQL.

4.3 Intégration OpenVAS

Afin de remplacer les données simulées par des vulnérabilités réelles, OpenVAS a été intégré au module VM selon le pipeline suivant :

1. Configuration de `gvm-cli` sur une machine Kali Linux.
2. Création d’une cible de scan (*target*) et d’une tâche (*task*) avec la configuration *Full and fast*.

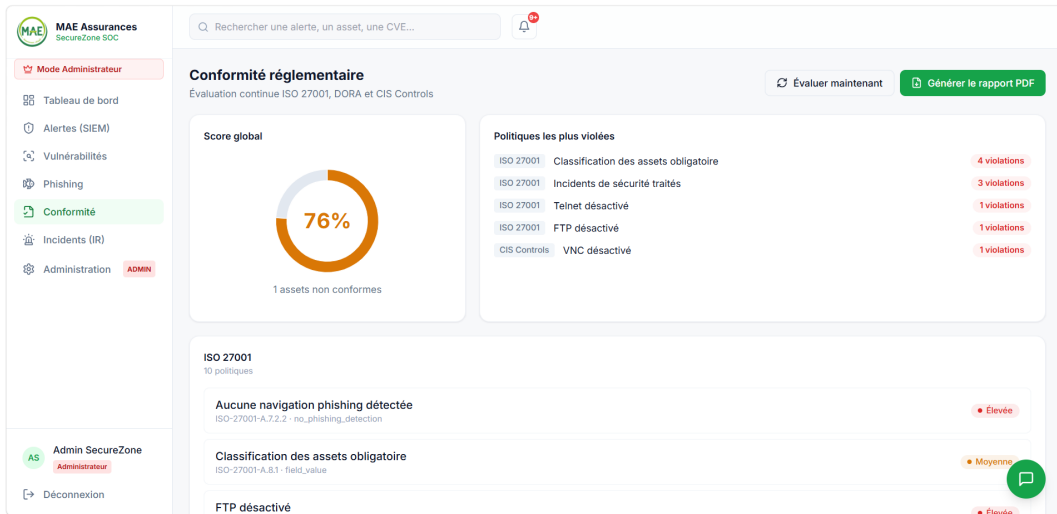


FIGURE 4.5 – Tableau de conformité ISO 27001 / DORA / CIS Controls

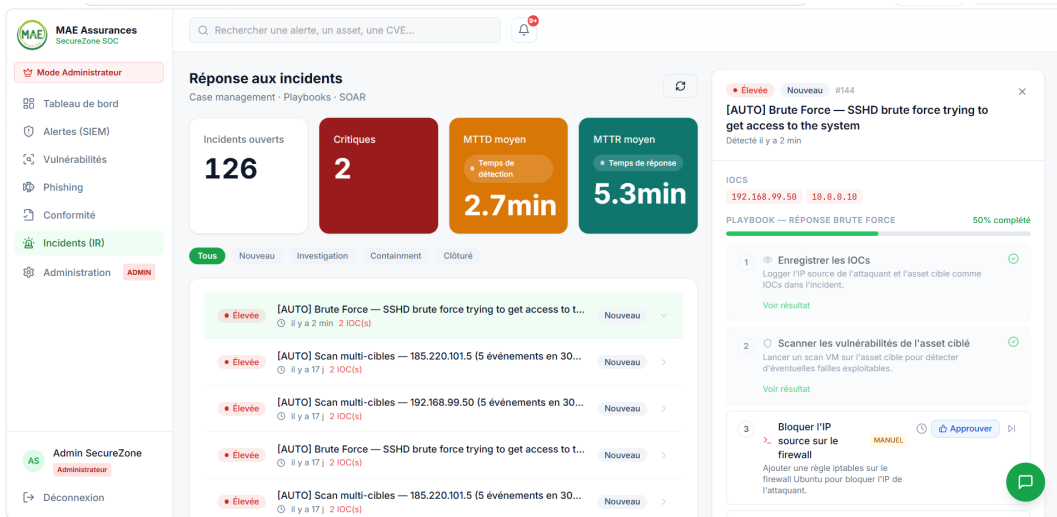


FIGURE 4.6 – Détail d'un incident avec playbook SOAR attaché

3. Lancement du scan contre une machine Metasploitable2.
4. Développement d'un endpoint `POST /vulnerabilities/import` côté backend pour recevoir les résultats.
5. Développement d'un script d'import s'authentifiant sur l'API SecureZone (JWT), récupérant le rapport OpenVAS, parsant le XML et envoyant les vulnérabilités extraites vers l'endpoint d'import.

Cette intégration a permis de valider le système sur des données réelles, révélant 363 vulnérabilités sur la machine cible.

4.4 Tests et Validation

Conclusion

Les interfaces réalisées couvrent l'ensemble des besoins fonctionnels identifiés au Chapitre 2. L'intégration réelle d'OpenVAS et la validation sur Metasploitable2 confirment la viabilité technique de SecureZone.

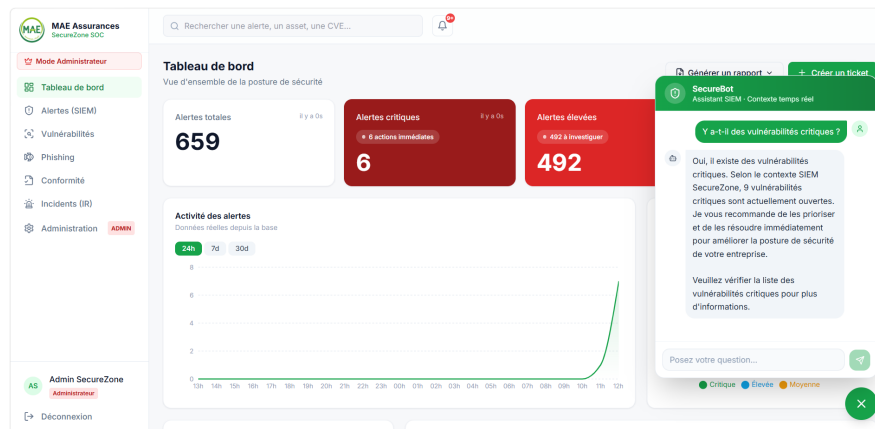


FIGURE 4.7 – Interaction avec l’assistant SecureBot

Méthode	Route	Description
GET	/vulnerabilities/	Lister les vulnérabilités
GET	/vulnerabilities/stats	Statistiques globales
GET	/vulnerabilities/{id}	Détail d’une vulnérabilité
PATCH	/vulnerabilities/{id}	Mettre à jour le statut
POST	/vulnerabilities/import	Import depuis un scanner externe

TABLE 4.1 – Endpoints du module Vulnerability Management

Composant testé	Statut	Détail
Collecte de logs (SIEM)	Validé	Logs normalisés et persistés
Détection d’anomalies (Isolation Forest)	Validé	Anomalies détectées sur logs de test
Corrélation (6 règles)	Validé	CR-001 déclenchée par simulation Hydra
Playbooks SOAR (4)	Validé	Brute Force, Phishing, Port Scan, Generic High Severity déclenchés correctement
Détection de phishing	Validé	Typosquatting et TLD suspects détectés, faux positif Google corrigé
Scan Nmap + OpenVAS	Validé	363 vulnérabilités détectées sur Metasploitable2
Import de vulnérabilités	Validé	Endpoint POST /import fonctionnel
Tableau de conformité	Validé	Score DORA/ISO 27001/CIS Controls calculé en temps réel
Assistant SecureBot	Validé	Réponses contextuelles correctes via Groq

TABLE 4.2 – Résultats des tests de validation

Chapitre 5

Limites et Perspectives

Plan

Introduction
5.1 Limites Actuelles
5.2 Perspectives
Conclusion

Introduction

Ce chapitre présente un bilan critique du système développé, identifiant ses limites actuelles ainsi que les perspectives d'amélioration envisagées.

5.1 Limites Actuelles

Limite	Sévérité	Explication
Séries temporelles du dashboard simulées	Moyenne	Pas encore d'endpoint d'agrégation historique
État de corrélation perdu au redémarrage	Moyenne	Fenêtres glissantes en mémoire (RAM)
ML nécessite 100+ événements	Faible	Mode passthrough sur base fraîche
Pas de page de gestion des actifs	Faible	Actifs créés uniquement par le scanner
Exceptions de conformité sans expiration	Faible	Champ de date présent mais non vérifié automatiquement
Pas de notifications email/Slack réelles	Faible	Action <code>send_notification</code> journalisée en base uniquement
MTTD non calculé automatiquement	Faible	Nécessiterait de connaître l'heure de début réelle de la menace

TABLE 5.1 – Limites actuelles de SecureZone

5.2 Perspectives

- Remplacer l'état de corrélation en mémoire par des ensembles triés Redis, permettant un déploiement multi-worker.
- Ajouter un endpoint d'agrégation historique pour remplacer les séries temporelles simulées du dashboard.
- Intégrer un job planifié vérifiant l'expiration des exceptions de conformité.
- Configurer des notifications réelles (SMTP, webhook Slack).
- Containeriser l'architecture avec Docker Compose pour faciliter le déploiement en production chez MAE.

Conclusion

Ce chapitre a dressé un bilan honnête des limites actuelles de SecureZone, inhérentes à un contexte de stage et de plateforme de démonstration, ainsi que les axes d'évolution vers un déploiement en production.

Conclusion Générale

Ce stage avait pour objectif la conception et le développement d'une plateforme de cybersécurité unifiée intégrant SIEM, détection de phishing, gestion des vulnérabilités, conformité réglementaire et réponse aux incidents. À travers ce travail, une solution complète nommée SecureZone a été réalisée au sein de MAE, répondant aux exigences de la réglementation DORA applicable aux compagnies d'assurance.

Le système développé intègre un moteur SIEM avec détection d'anomalies par Machine Learning et corrélation par règles, un module de détection de phishing heuristique, un moteur de gestion des vulnérabilités intégrant des scanners réels (Nmap, OpenVAS), un moteur de conformité évaluant automatiquement les contrôles DORA/ISO 27001/CIS Controls, ainsi qu'un moteur d'orchestration SOAR pilotant quatre playbooks de réponse. Les principaux objectifs fixés ont été atteints, notamment la mise en place d'une architecture backend/frontend complète et la validation du système sur des données réelles.

Sur le plan personnel, ce stage a constitué une opportunité enrichissante, permettant de développer mon autonomie et ma capacité à intégrer plusieurs technologies de cybersécurité au sein d'un même système.

Parmi les perspectives d'évolution figurent la migration de l'état de corrélation vers Redis, la containerisation de l'architecture avec Docker Compose, ainsi que le déploiement en environnement de production chez MAE.

Bibliographie

- [1] FastAPI Documentation. <https://fastapi.tiangolo.com>
- [2] Wazuh Documentation. <https://documentation.wazuh.com>
- [3] Digital Operational Resilience Act (DORA), Règlement (UE) 2022/2554.
- [4] Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. *IEEE ICDM*.
- [5] Greenbone / OpenVAS Documentation. <https://greenbone.github.io>
- [6] MITRE ATT&CK Framework. <https://attack.mitre.org>